

Dependently Typed World Models

Toward Proof-Carrying Learned Simulators

Jordan Rule

`jrul@uchicago.edu`

September 21, 2026

Abstract

A learned world model can be pictured as a latent geometry: many partial views woven together, a rule for their coherence, and a gate that authorizes action only when the pieces align. This paper proposes that dependent types offer a disciplined foundation for world models, because they embed evidence directly into state and bind every action to a formal proof of its preconditions. The result is a framework where learned predictive power meets explicit accountability.

1. Introduction

This paper proposes a framework for world models in which a decision arises only after composing many partial latent states and verifying their compatibility. We visualize the latent space as an n -dimensional product $\mathcal{Z} = Z_1 \times Z_2 \times \dots \times Z_n$, where each coordinate captures a different view of the situation: perception, memory, inference, or constraint. A decision is admissible only when a subset of these coordinates can be lawfully joined.

Formally, let $\mathbf{Formed}(e_i, z_i)$ assert that coordinate i is correctly formed from evidence e_i . The set of admissible states is the intersection

$$\mathcal{A} = \left\{ z \in \mathcal{Z} : \mathbf{Adm}(z) \Leftrightarrow \exists e_1, \dots, e_m. \bigcap_{i=1}^m \mathbf{Formed}(e_i, z_i) \right\}$$

A next action is typed by membership in $\mathcal{A}$:

$$a : \mathbf{Next}(z, \mathbf{Adm}(z)) \quad \text{only if} \quad z \in \mathcal{A}.$$

This structure has three properties. First, it makes admissibility a composite judgment: a state earns its authority by aligning many sources of evidence. Second, it prevents action from orphaned latent fragments. Third, it offers dependent types a concrete role: as a way to encode this alignment requirement into the type of an action itself, ensuring that the policy cannot be constructed without the precondition.

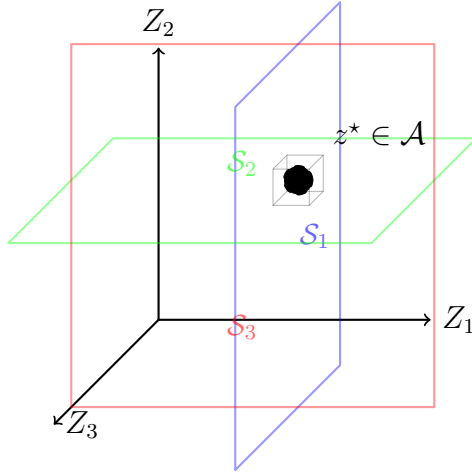


Figure 1: Three partial latent coordinates intersect at z^* , which lies in the admissible region $\mathcal{A}$. Each slice $\mathcal{S}_i$ represents states satisfying $\text{Formed}(e_i, z_i)$ for evidence e_i . Only their intersection is authorized to generate a next action.

Figure 1 presents this picture in three dimensions. The proposal is that this structure transfers naturally to Rocq, where such constraints can be formally verified and preserved during learning.

2. Background

2.1. World Models in Reinforcement Learning

A world model, in the narrow reinforcement-learning sense, is a learned surrogate for the dynamics of an environment. Formally one may write it as a map $f_\theta : \mathcal{Z} \times \mathcal{A} \rightarrow \mathcal{Z} \times \mathcal{R}$, sending a latent state and an action to a predicted successor state and reward. But that notation only hints at the broader intuition. A world model is a compressed, internal picture of how things change: what tends to follow what, which possibilities remain open, and which imagined futures are worth spending computation on.

The modern lineage is familiar. Dyna first made explicit the advantage of learning and planning in tandem [Sutton, 1990]. Ha and Schmidhuber’s World Models popularized the image of an agent dreaming inside its own latent space [Ha and Schmidhuber, 2018]. Dreamer and its successors showed that long-horizon imagination could become a practical engine for control [Hafner et al., 2020, 2021, 2023]. MuZero and EfficientZero demonstrated that one need not reconstruct the visible world in full detail in order to plan effectively [Schrittwieser et al., 2020, Ye et al., 2021].

JEPA extends the same family resemblance in a slightly different direction. Rather than learning to regenerate an entire input, a JEPA-style system learns to predict the representation of a withheld target view from an available context view [LeCun, 2022, Assran et al., 2023]. What matters is not visual fidelity but predictive sufficiency: the latent state should preserve the structure needed to

say something reliable about what lies beyond the current glimpse. In that sense, JEPA is not a departure from the world-model story so much as a sharpening of one of its oldest instincts.

Across these traditions, a recurring tension remains between *predictive skill* and *explicit meaning*. A model may be extraordinarily good at forecasting what happens next and still be silent about what sort of state it is claiming to inhabit, what sources of evidence it has mixed together, and what operational limits should govern its use. This gap is the starting point for our proposal.

2.2. Dependent Type Theory and the Rocq Proof Assistant

Dependent type theory [Martin-Löf, 1984, Coquand and Huet, 1988] begins from a simple but powerful thought: the description of an object may be allowed to mention the object itself. Instead of saying merely “this is a vector,” one may say “this is a vector of length n ,” and insist that n be part of the object’s type. A familiar example is $\text{Vec}(n)$, the type of vectors of exactly n elements. More generally, a value of type $\{x : A \mid P(x)\}$ is both a value in A and evidence that it satisfies $P(x)$.

For readers outside formal methods, the philosophical point matters more than the notation. In an ordinary program, one often constructs a value first and checks later whether it meets the relevant conditions. In a dependently typed program, one can require those conditions at birth. Some classes of mismatch never become runtime problems because the system refuses to construct the wrong object in the first place.

Rocq (formerly Coq) [The Coq/Rocq Development Team, 2024] implements the Calculus of Inductive Constructions [Coquand and Huet, 1988, Paulin-Mohring, 2015], a rich dependent type theory supporting:

- *Inductive types* and pattern matching;
- *Dependent records* (**Record**) that bundle values with their correctness conditions;
- *Proof terms* as first-class citizens of the same language as programs;
- *Extraction* to OCaml or Haskell, enabling certified programs to run outside the proof environment.

The MathComp library [Gonthier et al., 2013] provides mature foundations for constructive formalization in Rocq and practical proof engineering at scale.

2.3. Partial Views, Predictive Sufficiency, and Contracts

The core structure is a directed pipeline from partial states to admissible decisions. Let $\mathcal{Z} = (Z_1, Z_2, \dots, Z_n)$ be an n -dimensional latent space. For each dimension i , a witness function $h_i : \text{Formed}(e_i, z_i)$ asserts that coordinate z_i is a valid summary of evidence e_i .

The compositional rule is: if $h_1, \dots, h_m$ are witnesses for a subset of coordinates, and their evidence $e_1, \dots, e_m$ satisfies $\text{Compat}(e_i, e_j)$ for all i, j in the subset, then the product

$$z^\star = (z_1, \dots, z_m, \perp, \dots, \perp)$$

is admissible and can authorize action. The point is that not all latent states are action-worthy; only those composed from compatible evidence carry authority.

2.4. From JEPA Representations to Typed Planning

In many deployments, the central failure mode is not poor prediction in isolation but overconfident use of prediction in the wrong context. A model can be right on average and still dangerous in the moments that matter. The bridge we propose is therefore interface-level: keep JEPA as the statistical engine for partial-view prediction, and place a typed contract at the boundary where predictions become plans.

Under this view, a planner does not merely consume a latent state. It consumes a *checked* latent state, paired with explicit witnesses about provenance, coverage, or confidence regime. The point is not to freeze learning into rigid symbolism; it is to stop treating all latent states as equally action-worthy.

3. A Framework for Proof-Carrying World Models

3.1. Motivation

We want the latent space to feel less like a flat container and more like a surface with overlaps. Let $\mathcal{Z}$ be the carrier set of latent states. Let **Adm** mark the states that are safe to use, and let **Compat** say when two pieces can be viewed together. Then admissibility is not a label placed on the state after the fact; it is a composite judgment made from the pieces that formed it.

3.2. Proof-Carrying State

A proof-carrying state is a pair (z, h) where $z \in \mathcal{Z}$ and $h : \mathbf{Adm}(z)$. The proof term h is not a comment; it is part of the object. Construction and certification are inseparable.

Definition 3.1 (Proof-Carrying State). *Let $\mathcal{Z}$ be a set of latent states and $\mathbf{Adm} : \mathcal{Z} \rightarrow \mathbf{Prop}$. A proof-carrying state is a dependent pair (z, h) where $z : \mathcal{Z}$ and $h : \mathbf{Adm}(z)$.*

Formally, the type of all proof-carrying states is

$$\Sigma_{z:\mathcal{Z}} \mathbf{Adm}(z) = \{(z, h) : z \in \mathcal{Z}, h : \mathbf{Adm}(z)\}.$$

For world models, there is usually a raw latent z_0 proposed by perception or prediction, followed by a check step that produces a witness $h : \mathbf{Adm}(z_0)$ or fails. Only after that check does the latent become eligible for planning.

Definition 3.2 (Raw and Checked Latent). *Let $\mathcal{Z}$ be a latent space, $\mathcal{E}$ a space of evidence, and $\mathbf{Formed} : \mathcal{E} \times \mathcal{Z} \rightarrow \mathbf{Prop}$. A checked latent is a triple (z, e, h) where $z : \mathcal{Z}$, $e : \mathcal{E}$, and $h : \mathbf{Formed}(e, z)$.*

The distinction matters operationally. A raw latent says, “here is a candidate state.” A checked latent says, “here is a state, here is the evidence, and here is

the proof that the state correctly summarizes that evidence.” The proof is the authorization to use the state in planning.

In a JEPA setting, a latent z is formed from a context-target pair (c, t) via an encoder $\text{enc} : C \rightarrow Z$. Evidence e is the structured record of what the context contained and what was predicted. The formation judgment $(c, t) \Downarrow (z, e, h)$ holds when $z = \text{enc}(c)$ and $h : \text{Formed}(e, z)$. The witness h certifies that the latent is a faithful encoding of the evidence available in (c, t) .

Proposition 3.1 (Formation Under Evidence Constraints). *Suppose $(c_1, t_1) \Downarrow (z_1, e_1, h_1)$ and $(c_2, t_2) \Downarrow (z_2, e_2, h_2)$. If $\text{Compat}(e_1, e_2)$ holds, then the pair (z_1, z_2) can be combined into a joint state z_{12} with a composite witness*

$$h_{12} : \text{Formed}(e_1 \oplus_E e_2, z_{12}),$$

where $\oplus_E$ denotes evidence fusion.

3.3. Typed Next-Action Families

An action in this framework is not a proposition floating free of its justification. It is a term of a dependent type indexed by the proof-carrying state that authorizes it.

Definition 3.3 (Typed Next Action). *Let $\text{Next} : \mathcal{Z} \times \text{Adm}(\mathcal{Z}) \rightarrow \text{Type}$ be a family assigning to each proof-carrying state (z, h) a type of next actions. A next action is a term $a : \text{Next}(z, h)$.*

The dependence creates a feedback loop: one cannot emit an action of type $\text{Next}(z, h)$ without having already constructed the proof h . This is not a safety wrapper around a plan; it is built into the plan’s type.

In multi-agent settings, this structure is particularly valuable. Each agent proposes an action only from a proof-carrying state. When agents must negotiate a joint plan, they must first reconcile their evidence, then jointly construct a proof that the merged state is admissible. If reconciliation fails, the type system prevents the joint plan from existing.

Proposition 3.2 (Typed Action Consistency). *If $a_1 : \text{Next}(z_1, h_1)$ and $a_2 : \text{Next}(z_2, h_2)$, and if $\text{Compat}(e_1, e_2)$ holds for the evidence underlying h_1 and h_2 , then there exists a joint action $a_{12} : \text{Next}(z_{12}, h_{12})$ where (z_{12}, h_{12}) is the composite proof-carrying state.*

4. From Partial Views to Planning Contracts

4.1. JEPA and the Grammar of Missing Information

JEPA’s conceptual strength is that it takes missing information seriously. It asks what can be inferred from what is present, without pretending that hidden structure has been directly observed [LeCun, 2022, Assran et al., 2023]. For autonomous systems operating in the wild, this is the norm rather than the exception: data arrive late, sensors disagree, and collaborators report through uneven trust channels.

Dependent types offer a useful complement: they let us distinguish between “predicted” and “safely actionable” in the type of an object, not only in a policy note outside the model. This creates a clean handoff: JEPA supplies statistical abstraction; typed contracts govern admissible use.

4.2. Planning Under Limited Knowledge

When knowledge is partial, planning should degrade gracefully rather than fail silently or overreach. A typed interface can enforce this behavior by requiring that high-impact actions carry explicit witnesses that their preconditions are met under the current evidence profile. If those witnesses are absent, the plan construction itself fails.

This is not merely elegant formalism. It is operationally relevant in settings where autonomous systems must choose under ambiguity. A planner that cannot prove enough should return an abstention, a request for more context, or a lower-risk fallback policy. Typed construction gives these behaviors first-class status.

4.3. A Collaborative Visualization

The latent space is a product of coordinates. The admissible region is their intersection under the compatibility constraint. Figure 1 (in the Introduction) illustrates this in three dimensions: each colored slice represents a single evidence constraint, and their intersection marks the region from which action may be authorized.

An action in this framework is not a free-standing command but a term typed by the witnesses that the latent state is admissible. The effect is powerful: one cannot construct an action without simultaneously constructing a proof that its preconditions are met.

4.4. Immediate Alignment Value in Multi-Agent Settings

The safety advantage is concrete. In multi-agent environments, failures often arise not from one bad prediction but from a mismatch between pieces. Typed composition makes that mismatch visible before action.

The result is still learned and adaptive, but it is less willing to improvise beyond its evidence.

5. Illustrative Rocq Formalizations

5.1. A Tiny Example of Proof-Carrying State

Before introducing anything more ambitious, it helps to see the smallest possible case: a checked state whose horizon is certified by a witness.

Listing 1: A minimal checked state in Rocq

```
From Coq Require Import List Arith.
Import ListNotations.
```

```

Record CheckedRollout := {
  horizon : nat;
  actions : list nat;
  horizon_matches : length actions = horizon
}.

Definition single_action (a : nat) : CheckedRollout :=
  {| horizon := 1;
    actions := [a];
    horizon_matches := eq_refl |}.

```

The field `horizon_matches` is not a runtime assertion but a proof term. It is discharged by `eq_refl` at the point of construction, confirming that the singleton list `[a]` has length 1. A `CheckedRollout` with mismatched horizon cannot be constructed; the attempt would produce a type error. This is the paper’s basic promise in miniature: if a state claims to summarize a horizon, that claim can be made part of the object rather than an informal comment about it.

5.2. Typed Next-Action Sketch

The second sketch makes the dependence on the latent state explicit. A planner does not emit an action in the abstract; it emits an action licensed by the state and by the witness that the state is admissible.

Listing 2: A dependent next-action family in Rocq

```

Section NextAction.
  Variable State Action : Type.
  Variable Adm : State -> Prop.
  Variable Next : State -> Type.

  Record Planner := {
    propose : forall s, Adm s -> Next s
  }.
End NextAction.

```

The point is not to build a full planning stack in a few lines. It is to show the shape of the interface: the next action is indexed by the state that authorizes it, so the latent space and the policy are never fully separable.

6. Discussion and Research Directions

6.1. Alignment-Aware Planning

The alignment problem for learned agents [Russell, 2019, Gabriel, 2020] is, in part, a problem of specification: how does an agent know that the actions it is

considering are within the scope of what it is supposed to do? Current approaches rely heavily on reward shaping, RLHF [Christiano et al., 2017, Bai et al., 2022b], and constitutional constraints [Bai et al., 2022a], all of which are intrinsically extrinsic—they surround the model with evaluative signals rather than embedding constraints within its representational structure.

A proof-carrying world model offers a complementary approach. By requiring that high-stakes action types carry typed witnesses of their preconditions, the planning procedure is structurally prevented from producing plans that violate stated constraints—not because a guard intercepts the output, but because the output type cannot be constructed without the required evidence. The difference is analogous to the difference between a runtime bounds check and a statically verified array access: the latter eliminates a class of error by construction, before execution begins.

We envision a planning module whose action type is parameterized by an evidence profile ϵ : an action of type $\mathbf{Action}(\epsilon)$ may only rely on assumptions justified by ϵ . Confidence escalation then becomes a type error.

The JEPA connection matters here as well. If a predictive model is trained to infer withheld structure from partial context, the question naturally follows: what kinds of context are sufficient for which kinds of action? Dependent types offer a way to state that question sharply. A latent prediction can be treated not as a free-floating hint, but as an object whose admissible uses are tied to the evidence from which it was formed.

6.2. Multi-Agent Trust and Fragment Provenance

In multi-agent settings, observations arrive from diverse sources with varying degrees of trust. A dependently typed world model can represent this by annotating observation histories with *provenance types*: a history of type $\mathbf{Hist}(\tau)$ was collected under trust level τ , and the type system can enforce that a planner operating under trust level τ' may only act on histories whose provenance satisfies $\tau' \preceq \tau$ for some trust ordering $\preceq$.

This makes cross-provenance contamination detectable at the type level—a property with direct practical significance whenever agents interact with adversarially constructed inputs or aggregated data of heterogeneous origin.

7. Related Work

World models. Ha and Schmidhuber’s World Models [Ha and Schmidhuber, 2018] helped establish the “dream-then-act” paradigm for visual planning. The DreamerV1–V3 series [Hafner et al., 2020, 2021, 2023] demonstrated that continuous control and Atari tasks can be mastered through pure imagination. MuZero [Schrittwieser et al., 2020] showed that a latent model without ground-truth reconstruction suffices for search-based planning. EfficientZero [Ye et al., 2021] extended this to extreme sample efficiency. None of these architectures address typed invariants on the model interface.

JEPA and predictive representation learning. LeCun’s program for autonomous machine intelligence [LeCun, 2022] argues that useful intelligence should be grounded in predictive world models that operate in latent space rather than through exhaustive reconstruction. I-JEPA [Assran et al., 2023] makes this idea concrete by learning to predict target embeddings from context embeddings. Our use of typed predictive contracts is meant to be complementary: JEPA supplies a modern account of how predictive latent structure may be learned, while Rocq supplies a way to state, with precision, what such predictions are allowed to authorize in planning.

Formal methods and learning. Seshia et al. [Seshia et al., 2018] survey formal specification and verification techniques applicable to learning-enabled systems, emphasizing runtime monitors and assume-guarantee contracts. Neural network verification [Katz et al., 2019, Liu et al., 2021] focuses on robustness of fixed networks. CompCert [Leroy, 2009] and seL4 [Klein et al., 2009] demonstrate that large software systems can be given machine-checked correctness proofs; our work proposes that learned interfaces can participate in such developments.

Alignment and specification. Russell [Russell, 2019] frames alignment as a problem of misspecified objectives. Christiano et al. [Christiano et al., 2017] and Bai et al. [Bai et al., 2022b,a] address alignment through human feedback and constitutional constraints. Gabriel [Gabriel, 2020] provides a conceptual taxonomy. Our proposal is orthogonal: rather than specifying objectives more accurately, we propose to specify the operational interface of the model more precisely.

8. Conclusion

We have argued that world models become easier to reason about when they are understood as accountable constructions rather than opaque latent containers. JEPA reminds us that useful predictive structure can be learned from partial views. Dependent type theory adds a discipline for saying what those predictions license: when a state is admissible, when evidence is sufficient, and when a plan should abstain.

The proposal is not that proofs replace learning. A well-typed world model may still be empirically wrong about the world. The proposal is narrower and, for that reason, practical: that learned internal models can carry clearer commitments about admissibility, provenance, and use in multi-agent systems. If a system is going to imagine on our behalf, it is reasonable to ask that its imagination come with explicit terms.

References

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning

- from images with a joint-embedding predictive architecture. *arXiv preprint arXiv:2301.08243*, 2023.
- Yuntao Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022a.
- Yuntao Bai et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022b.
- Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2–3):95–120, 1988.
- Iason Gabriel. Artificial intelligence, values, and alignment. *Minds and Machines*, 30:411–437, 2020.
- Georges Gonthier et al. A machine-checked proof of the odd order theorem. In *International Conference on Interactive Theorem Proving (ITP)*, pages 163–179, 2013.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.
- Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations (ICLR)*, 2021.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Guy Katz, Derek A. Huang, Duligur Ibeling, et al. The marabou framework for verification and analysis of deep neural networks. In *Computer Aided Verification (CAV)*, pages 443–452, 2019.
- Gerwin Klein et al. seL4: Formal verification of an OS kernel. In *ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.
- Yann LeCun. A path towards autonomous machine intelligence. *OpenReview preprint*, 2022.
- Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- Changliu Liu, Tomer Arnon, Christopher Lazarus, Christopher Strong, Clark Barrett, and Mykel J. Kochenderfer. Algorithms for verifying deep neural networks. *Foundations and Trends in Optimization*, 4(3–4):244–404, 2021.

- Per Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- Christine Paulin-Mohring. Introduction to the calculus of inductive constructions. In David Delahaye and Bruno Woltzenlogel Paleo, editors, *All about Proofs, Proofs for All*. College Publications, 2015.
- Stuart Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, 2019.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering atari, go, chess and shogi by planning with a learned model. In *Nature*, volume 588, pages 604–609, 2020.
- Sanjit A. Seshia, Dorsa Sadigh, and S. Shankar Sastry. Formal specification for deep neural networks. In *Automated Technology for Verification and Analysis (ATVA)*, pages 20–34, 2018.
- Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning (ICML)*, pages 216–224, 1990.
- The Coq/Rocq Development Team. The rocq proof assistant reference manual. <https://rocq-prover.org/>, 2024.
- Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.